

Predicting Defective Code Clones in Autonomous Driving Software

Chenyi Zhou, Ran Mo*, Weijun Hu
School of Computer Science
Central China Normal University
Wuhan, China

Abstract—In recent years, with the rapid development of automated driving technology, the safety and quality of automated driving software have become important concerns. Prior studies have shown that code clones are prevalent in autonomous driving software and significantly impact software quality, maintenance costs, and module independence. Although code cloning can improve development productivity, they are more likely to introduce co-modifications and software defects, which leads to high detection and fixing costs. In this paper, we propose a novel approach to predict defective code clones in autonomous driving software by focusing specifically on clone code snippets as prediction targets. We designed a comprehensive set of nine code metrics characterizing clone snippets from three dimensions: code size, complexity, and historical modifications. Using six machine learning algorithms, we built prediction models and evaluated them on two representative L4 open-source autonomous driving platforms: Apollo and Autoware. Our experimental results demonstrate that the developed prediction models achieve good performance with accuracy rates ranging from 82.7% to 94.4%. Through principal component analysis, we identified that the number of added lines, modification frequency, and clone line count are the most significant factors contributing to defect proneness in code clones. We believe that this work enables developers to locate defective code clones more precisely. Thus, they can improve testing efficiency, reduce bug fix costs, and ultimately improve the quality and reliability of autonomous driving software.

Index Terms—Automated Driving Software, Clone Code Snippet, Defective Clone Prediction

I. INTRODUCTION

With the rapid development of automated driving technology, it has attracted considerable attention from both academic researchers and industry practitioners [1]. As complex software integrating multiple tasks and often deployed in safety-critical environments, it is essential to ensure the safety and quality of automated driving software. Recent studies [2], [3] have indicated that code clones (i.e. the identical or similar code fragments in different places of programs) are prevalent in autonomous driving software and have significantly negative impacts on software quality, maintenance costs, and module independence. Although code clones can speed up the development process, they are also more likely to induce co-modifications and software defects, which may lead to high costs for detection and fixes. Therefore, it is important to locate potential defective code clones before the software is deployed.

Software defect prediction is one of the most active topics in software engineering. It aims to identify software units that are prone to defects at an early stage. Accurate prediction can help development teams effectively allocate software testing resources by focusing on defect-prone units, thereby reducing inspection costs and improving testing efficiency. In the past decades, researchers have conducted defect prediction at a diverse range of targeted objects, including files, methods, and classes. However, there is little work has been done on building prediction models that focus on defective code clones.

According to previous studies [2], [3], code clones occur commonly in autonomous driving software. Such clones have reduced the maintainability of autonomous driving software by causing modification inconsistency and propagating defects. Moreover, the defects involved in code clones are difficult to fix since these defects should be addressed in all cloned code fragments. Therefore, in this study, we proposed a novel perspective that takes code clones as the targets for defect prediction. We attempt to accurately identify defective and non-defective clone codes in autonomous driving software.

To proceed with our study, we first proposed a suite of clone-based features. On this basis, we then leveraged the machine learning algorithm to build prediction models for defective code clones. Through our experiments on two representative L4 open-source automated driving software, Apollo [4] and Autoware [5], we have found that our developed prediction models can achieve good predictive performance with accuracy rates of 82.7% - 94.4%. Additionally, we found that the number of added lines is the most significant feature indicating defective code clones. We believe that our work would enable developers to more precisely locate defective code clones, which in turn enhances the efficiency of testing and bug fixing and ultimately improves the quality and reliability of autonomous driving software.

The main contributions of our work are as follows:

- 1) We propose a novel approach to defect prediction by focusing specifically on clone code as prediction targets in autonomous driving software. Meanwhile, our built prediction models can achieve good performance (82.7 - 94.4% accuracy).
- 2) We design a comprehensive set of code metrics from three dimensions: code size, complexity, and historical modifications, to effectively characterize cloned code snippets.
- 3) We identify key characteristics of defective clone code

*Corresponding Author

snippets through principal component analysis, revealing that the number of added lines, modification times, and clone line count are primary factors contributing to defect proneness.

The rest of the article is structured as follows: Section II provides background and related work. Section III describes the design of our empirical study. Section IV presents our evaluation methodology, analysis, and results. Section V discusses the threats to the effectiveness of our work. Section VI concludes.

II. BACKGROUND AND RELATED WORK

A. Software Defect Prediction

In the past few decades, software defect prediction has been a very popular research direction in the field of software engineering. Numerous researchers have conducted studies targeting a wide range of subjects.

Yan et al. [6] centered on effort-aware file-level defect prediction. They replicated a study on the PROMISE dataset and found differences in the performance of unsupervised and supervised models for predicting defective files.

Hata et al. [7] focused on detecting fault-prone modules by using large-scale text features from source code. The authors applied Naive Bayes and logistic regression models for prediction. Through the experiments on five projects, they demonstrated that text features improved detection accuracy compared to traditional metrics.

Mo et al. [8] used code metrics and history measures to predict bug-prone methods, and demonstrated that method-level analysis had the potential to save effort in code reviews and inspections. Additionally, they analyzed the metrics' prediction power and showed that history metrics outperformed code metrics in identifying bug-prone methods.

Yin et al. [9] introduced LineDef, a prediction method that used Graph Convolutional Networks(GCNs) to capture code contexts for predicting defective files and lines. LineDef consists of three components: token embedding, graph extraction, and multi-granularity defect prediction. It mapped tokens to vectors, extracted line-level and token-level graphs by using a sliding window, and applied GCN layers and attention mechanisms to predict defects at both file and line levels. The study demonstrated the effectiveness of LineDef in extracting comprehensive information from code lines for defect prediction.

B. Automated Driving Software

The automated driving software plays an important role in handling core tasks, such as perception, planning, control, etc. To improve the quality and safety of autonomous driving, many researchers have carried out numerous studies.

Chitnis et al. [10] focused on ensuring the safety and quality of autonomous driving software by addressing redundancy, architecture, interference-free operation, and diagnostics. They proposed solutions for each aspect to meet ISO 26262's ASIL requirements, enhancing software reliability and preventing systematic faults.

Gómez-Huélamo et al. [11] presented a ROS-based modular architecture for an autonomous driving stack and validated it for safety and reliability. Their approach ensures robustness against non-trivial traffic scenarios and adverse environmental conditions, enhancing the reliability of autonomous vehicles by addressing dynamic challenges effectively.

Cheng et al. [12] employed CodeQL to examine vulnerabilities in key ADS projects, highlighting issues such as CWE-190 and CWE-20. Their research emphasizes the importance of incorporating static code analysis into ADS development to bolster security. This strategy is essential for maintaining the safety and reliability of autonomous driving software.

Despite numerous studies on autonomous driving software, defect prediction for this specialized software has received relatively little attention. Although many researchers have proposed software defect prediction models, studies focusing on clone code snippets remain unexplored. Prior studies [2], [3] showed that code clones are common in autonomous driving software and could introduce subtle and hard-to-fix defects. In our work, we proposed prediction models for predicting defective code clones in autonomous driving software. This approach helps developers locate defective code clones more efficiently, improves testing efficiency, reduces costs, and ultimately further improves the quality of autonomous driving software.

III. STUDY DESIGN

In this work, we focus on developing effective models to identify defective clone code snippets in autonomous driving software. Figure 1 shows the overall experimental procedures from clone code snippets gathering to model creation. We will next introduce each stage in detail.

A. Studied Projects

We chose two well-known autonomous driving software: Apollo [4] and Autoware [5] as our subjects. Since they are the only open-source projects that support Level 4 autonomous driving and have been adopted by numerous companies globally. We first used git to clone two projects locally. Table I provides an overview of the basic information for each project. The LOC column indicates the range of lines of code across different versions of the project. The File column shows the range of the number of files in each project across versions. The Commit column reflects the number of commits extracted from the project's revision history.

TABLE I
BASIC FACTS OF STUDIED PROJECTS

Project	Versions	LOC	File	Commit
Apollo	1.0.0-8.0.0	35,791-446,965	343-3,874	19,852
Autoware	1.0.0-1.10.0	131,222-193,443	695-1,320	3,794

Considering that the two projects are primarily written in C++, we used Simian¹ to identify code clones for each

¹<https://simian.quandarypeak.com/>

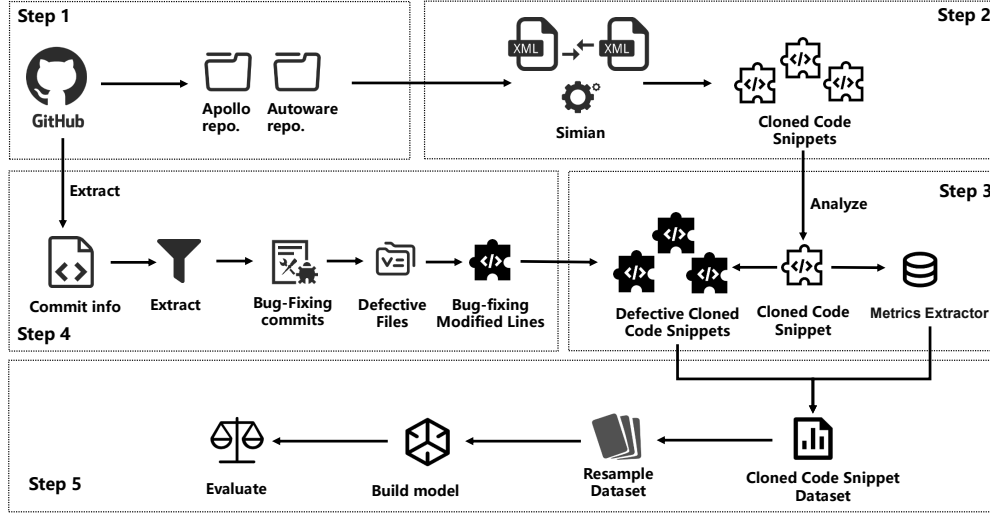


Fig. 1. EXPERIMENTAL FLOW CHART

autonomous driving software. Simian is a powerful text-based code clone detection tool that has been widely used in detecting code clones within C++ codebases. To achieve high recall and accuracy, we then configured the parameters following the recommendations of Ragkhitwetsagul et al. [13].

B. Feature Extraction

Clone-based Metrics. In this paper, we consider code clones as the targets of defect prediction. Therefore, we designed a set of code metrics that are directly extracted from the cloned code snippets in autonomous driving software. As shown in Table II. These metrics can effectively characterize the clone code snippets from three dimensions: code size, code complexity, and historical modification. Based on the detection results of code clones, we directly calculated the number of cloned code lines (C1). Additionally, we developed a tool to automate the calculation of four metrics (C2-C5), including the number of unique operators, the number of unique operands, the number of total operators, and the number of total operands, which can effectively characterize the complexity and structure of the clone code snippets. To better uncover the historical modification patterns of each clone code snippet, we calculated the number of times it had been modified, the number of lines added and deleted, and the number of unique authors who participated in these changes (C6-C9). These metrics offer insights into code complexity and historical features of code clones. For your reference, we have shared the link to our tool: <https://github.com/1209613917Alex/clone-code-snippets-predictor>

C. Class Labeling

In this case, we attempt to build defect prediction models to perform binary classification. Thus, we need to identify whether a code clone snippet is defective or non-defective and label it accordingly.

TABLE II
AN OVERVIEW OF CLONE-BASED METRICS

Index	Metric Name
C1	Number of Clone Lines
C2	Number of Unique Operators
C3	Number of Unique Operands
C4	Number of Total Operators
C5	Number of Total Operands
C6	Number of Authors
C7	Number of Modified Times
C8	Number of Added Lines
C9	Number of Deleted Lines

Defective code clone: We first downloaded all the commits from the project repository to extract all modification information from the project's history. Based on the defect dataset from prior studies [2], [3], we collected all defect-fixing commits, that is, the commits specifically addressing bug repairs. Then we extracted the modified lines, including the start line and end line, from the defect-fixing commits. If any part of a clone code snippet intersects with the modified lines identified in the bug-fixing commits, we mark this code clone as defective. By mining these defect-fixing commits, we can label all defective code clones. For clone code snippets that are not included in the modified lines of such bug-fixing commits, we marked them as non-defective, assuming they were not related to any known bug fixes.

D. Machine Learning Algorithms

Machine learning algorithms have demonstrated significant potential in constructing defect prediction models. In this work, we employed six well-established supervised learning methods to predict defective code clones:

We implemented these algorithms using Weka², a widely

²<https://ml.cms.waikato.ac.nz/weka>

TABLE III
ALGORITHM DESCRIPTIONS

Algorithm	Description
Random Forest (RF)	An ensemble technique that combines predictions from multiple decision trees to improve classification accuracy.
K-Nearest Neighbor (KNN)	A non-parametric, lazy-learning algorithm that classifies new samples by measuring similarity to nearby training data points.
Decision Trees (DT)	A non-parametric model that learns hierarchical decision rules to partition data into class labels.
Support Vector Machine (SVM)	A discriminative classifier defined by an optimal separating hyper-plane that maximizes margin between classes.
Multilayer Perceptron (MLP)	A feedforward artificial neural network with at least three layers, designed to simulate biological neuron interactions for complex pattern recognition.
Logistic Regression (LR)	A versatile model for classification tasks that uses a logistic function to map input features to binary class probabilities.

used machine learning tool, and maintained their default parameters to ensure reproducibility and fair comparison.

IV. EVALUATION

A. Research Questions

RQ1: *What is the proportion of defective clone code snippets in autonomous driving software?*

In this question, we extracted all the clone code snippets and completed the task of labeling. Next, we calculated the proportion of defective code clones in each of the studied projects to analyze the possibility of defects in clone code snippets.

RQ2: *Can we build a model to effectively predict the defective code clones in autonomous driving software?*

Using our proposed clone-based metrics, we applied machine learning algorithms to build models for predicting defective code clones. In this question, we examined whether the built models can accurately identify defective code clones in autonomous driving software.

RQ3: *What are the key characteristics of code clones that make them more prone to defects?*

In this question, we leveraged principal component analysis to calculate the weight of each metric we proposed, aiming to better understand the key characteristics of code clones that contribute the most to their defect-proneness. The results could help developers identify high-risk clone patterns and implement targeted fixing strategies.

B. Experiments and Results

RQ1: *What is the proportion of defective clone code snippets in autonomous driving software?*

To answer this question, we first count the total number of code clones in each autonomous driving software. Then we

calculated the ratios of defective clones to the total number of clones.

Table IV presents the experimental results. Column NTC shows the number of total code clones. Column NDC shows the number of defective code clones, and column PDC shows the ratio of NDC to NTC. For the Apollo project, we identified 5,468 code clones from version 1.0.0 to 7.0.0, and 551 of them are defective. Then the proportion of defective clone code snippets is 0.10.

Considering two software projects together, there are 8,670 code clones, and 1008 of them are defective. The average proportion of defective snippets in the two projects is about 12%. This non-trivial proportion indicates that developers cannot overlook the management and review of code clones in autonomous driving software. It reveals the potential risk that code clones pose to the overall quality and reliability of autonomous driving software.

TABLE IV
STATISTICS OF CODE CLONES

	NTC	NDC	PDC
Apollo	5,468	551	0.10
Autoware	3,202	456	0.14
Total	8,670	1,008	0.12

RQ2: *Can we build a model to effectively predict the defective clone code snippets in autonomous driving software?*

For each code clone, we extracted metrics mentioned in Table II as the features. Then we applied machine learning algorithms to build prediction models. To avoid biases due to imbalanced data, we used SMOTE [14] to equalize samples. To more effectively evaluate the models' performance, we employed a ten-fold cross-validation method to train and evaluate models [15]. This can avoid overfitting and ensure the stability and generalization ability of the model across different subsets of data. To evaluate the performance of prediction models, we utilized five widely accepted performance metrics: Accuracy, F-measure [16], precision, recall [17], and Area Under the Curve (AUC) [18].

Table V shows the performance of our prediction models. Taking project Apollo as an example, we find that when using the Random Forest (RF) algorithm to build the model, it shows good performance: accuracy is 89.3%, F-measure is 0.88, precision is 0.89, recall is 0.89 and AUC is 0.86. When applying the K-Nearest Neighbor (KNN) algorithm on Autoware, the five performance measures are respectively 90.4%, 0.90, 0.90, 0.90 and 0.86, which also shows decent performance. In general, the models built on using code-based metrics have good prediction ability. All accuracy rates exceed 83%, with the majority (8 out of 12) surpassing 85%. For the F-measure, most of the models (9 out of 12) achieve values larger than 0.8. Precision values range from 0.79 to 0.94, still suggesting good performance. And all recall values are larger than 0.83. In terms of the AUC measure, all values exceed 0.5,

TABLE V
PERFORMANCE COMPARISON OF ML ALGORITHMS IN APOLLO AND AUTOWARE

ML	Apollo					Autoware				
	ACC	F - m	Precision	Recall	AUC	ACC	F - m	Precision	Recall	AUC
RF	89.3%	0.88	0.89	0.89	0.86	94.4%	0.94	0.94	0.94	0.98
KNN	83.7%	0.83	0.83	0.84	0.64	90.4%	0.90	0.90	0.90	0.86
DT	86.2%	0.85	0.85	0.86	0.77	92.1%	0.92	0.92	0.92	0.93
SVM	86.2%	0.82	0.87	0.86	0.58	82.7%	0.78	0.86	0.83	0.61
MLP	83.3%	0.77	0.79	0.83	0.54	88.9%	0.89	0.89	0.89	0.92
LOG	83.6%	0.78	0.80	0.84	0.64	85.0%	0.83	0.84	0.85	0.90

and most of them (7 out of 12) are larger than 0.7, indicating promising prediction performance, which further confirms the models' ability.

The above evaluation results provide evidence that our proposed prediction models are highly effective in predicting defective code clones in autonomous driving software. By leveraging the metrics derived from clone code snippets, we were able to employ machine learning algorithms to build robust prediction models. These models have demonstrated good performance in terms of Accuracy, F-measure, precision, recall, and AUC.

RQ3: *What are the key characteristics of clone code snippets that make them more prone to defects?*

Principal component analysis (PCA) is an unsupervised learning method that only focuses on feature relationships. Hence we first dropped the label column from the dataset to ensure that our analysis is free from label influence [19]. We used PCA to extract the principal components, which are the new features that represent the data in a lower-dimensional space. These components are linear combinations of the original features and are ordered by the amount of variance they explain, from most to least. To determine the importance of each original feature, we calculated the weights of the original features in the new components using the loadings matrix. The weight W_j of the L_{ji} feature is calculated as the average absolute value of its loadings across all principal components, as shown in formula (1).

$$w_j = \frac{1}{k} \sum_{i=1}^k |L_{ji}| \quad (1)$$

k is the number of principal components, and L_{ji} is the loading of the j -th feature on the i -th principal component. The weight w_j indicates the importance of the j -th feature in capturing the variance in the data. By analyzing these weights, we can identify which features are most influential in representing the variability of the dataset. Features with higher weights play a more significant role in the structure of the data, which helps in understanding the key characteristics of defective clone code snippets.

Table VI presents the weights assigned to each feature in our analysis. The "Metric Name" column lists the features utilized in predicting defective code clones. The "Apollo-W" column displays the weight results for the Apollo project,

TABLE VI
WEIGHT OF EACH FEATURE

Metric Name	Apollo-W	Autoware-W	Average
Number of Clone Lines	0.19	0.20	0.195
Number of Unique Operators	0.13	0.14	0.135
Number of Unique Operands	0.14	0.14	0.140
Number of Total Operators	0.15	0.14	0.145
Number of Total Operands	0.14	0.14	0.140
Number of Authors	0.14	0.13	0.145
Number of Modified Times	0.23	0.18	0.205
Number of Added Lines	0.24	0.21	0.225
Number of Deleted Lines	0.17	0.19	0.180

while the "Autoware-W" column shows the corresponding weights for the Autoware project. The "Average" column provides a comprehensive overview of the results.

The experimental results show that the number of added lines is the most significant factor, indicating that an increased number of added lines within a clone code snippet during a modification correlates with a higher likelihood of introducing defects. Following this, the frequency of modifications to a defective clone code snippet ranks second, suggesting a strong relationship between the number of modifications and the potential for defects. Lastly, the number of clone lines, which represents the size of the snippets, ranks third, further highlighting the significant impact of the size of clone code snippets on defect occurrence.

These findings present the importance of focusing on high-weight indicators when working with code clones in autonomous driving software. By understanding these key characteristics, developers can proactively mitigate risks of code clones and enhance the overall quality and reliability of autonomous driving systems.

V. DISCUSSION

In this study, we employed two widely used technical tools, Simian and Weka, to facilitate the experimental process. To ensure the reproducibility of the experiments, we used the default parameter settings for both tools to obtain objective and accurate experimental results. However, we also noted that using these two tools with different parameter settings might affect the results. Specifically, different parameters in Simian could influence the detection results of code clones, while different parameters in Weka may impact the performance of the model.

In our experiment, we proposed a set of code metrics based on clone code snippets. Although we only selected nine metrics, these metrics comprehensively characterize the information of clone code snippets from three dimensions: code size, code complexity, and historical modification of clone code snippets, demonstrating strong representativeness. This is also evidenced by their good performance when used to construct models with machine learning algorithms. However, we are aware that in future work, we could attempt to employ deep learning methods, particularly focusing on the semantic and syntactic aspects of code clones, to explore the deeper relationships between clone code snippets and software defects. This approach may further enhance the accuracy of predicting defective clone code snippets in autonomous driving software.

Regarding external validity, our research only centered on two autonomous driving software. However, these two studied projects are the most representative L4 open-source autonomous driving software. Autoware has achieved widespread global adoption across multiple vehicle platforms through extensive industry collaborations worldwide [20]. Similarly, Apollo supports a broad ecosystem of international partners leveraging its technology framework [21]. Through our research, With over 1,000 defect clone code snippets collected and strong model prediction results, we believe that our findings are likely to be representative and can be generalized to other automated driving software systems.

VI. CONCLUSION

In this paper, we proposed a novel approach to predict defective code clones in autonomous driving software. Specifically, we designed a suite of clone-based metrics. Using them as features, we applied machine learning algorithms to build defect prediction models.

Through our comprehensive empirical study on two representative L4 open-source autonomous driving platforms, Apollo and Autoware, we have found that 1) code clones are prevalent in autonomous driving software, with approximately 12% of clone code snippets being defective. This non-trivial proportion indicates the importance of proactive management and review of code clones in safety-critical autonomous driving systems. 2) Our built prediction models can achieve impressive performance, with accuracy rates ranging from 82.7% to 94.4%. The Random Forest algorithm performed exceptionally well, achieving 89.3% accuracy on Apollo and 94.4% on Autoware. 3) Our principal component analysis revealed key characteristics that make code clones more prone to defects. The number of added lines emerged as the most significant factor, followed by modification frequency and clone line count. These insights could provide valuable guidance for developers to identify high-risk clone patterns.

In conclusion, we believe our work can provide a valuable approach for the autonomous driving software development, which offers both insights into the characteristics of defective code clones and practical prediction models that can effectively identify such clones.

ACKNOWLEDGE

This work is supported by the Hubei Provincial Natural Science Foundation of China under the grant No. 2025AFB969.

REFERENCES

- [1] A. Tampuu, T. Matiisen, M. Semikin, D. Fishman, and N. Muhammad, "A survey of end-to-end driving: Architectures and training methods," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 4, pp. 1364–1384, 2020.
- [2] R. Mo, Y. Jiang, W. Zhan, D. Wang, and Z. Li, "A comprehensive study on code clones in automated driving software," in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Sept. 2023, pp. 1073–1085.
- [3] Y. Jiang, R. Mo, W. Zhan, D. Wang, Z. Li, and Y. Ma, "Leveraging modular architecture for bug characterization and analysis in automated driving software," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 4, pp. 1–31, 2025.
- [4] Baidu, "Apolloauto," <https://github.com/ApolloAuto/apollo>, last accessed 2025/4/9.
- [5] TheAutowareFoundation, "Autoware," <https://github.com/autowarefoundation/autoware>, last accessed 2025/4/9.
- [6] M. Yan, Y. Fang, D. Lo, X. Xia, and X. Zhang, "File-level defect prediction: Unsupervised vs. supervised models," in *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, Nov. 2017, pp. 344–353.
- [7] H. Hata, O. Mizuno, and T. Kikuno, "Fault-prone module detection using large-scale text features based on spam filtering," *Empirical Software Engineering*, vol. 15, pp. 147–165, 2010.
- [8] R. Mo, S. Wei, Q. Feng, and Z. Li, "An exploratory study of bug prediction at the method level," *Information and software technology*, vol. 144, p. 106794, 2022.
- [9] S. Yin, S. Guo, H. Li, C. Li, R. Chen, X. Li, and H. Jiang, "Line-level defect prediction by capturing code contexts with graph convolutional networks," *IEEE Transactions on Software Engineering*, 2024.
- [10] K. Chitnis, M. Mody, P. Swami, R. Sivaraj, C. Ghone, M. G. Biju, B. Narayanan, Y. Dutt, and A. Dubey, "Enabling functional safety asil compliance for autonomous driving software systems," in *Electronic Imaging*, vol. 29, 2017, pp. 35 – 40.
- [11] C. Gómez-Huélamo, A. Díaz-Díaz, J. Araluce, M. E. Ortiz, R. Gutiérrez, F. Arango, A. Llamazares, and L. M. Bergasa, "How to build and validate a safe and reliable autonomous driving stack? a ros based software modular architecture baseline," in *2022 IEEE Intelligent Vehicles Symposium (IV)*, 2022, pp. 1282 – 1289.
- [12] W. Cheng, Z. Li, P. Liang, R. Mo, and H. Liu, "Unveiling security weaknesses in autonomous driving systems: An in-depth empirical study," *Information and Software Technology*, vol. 182, p. 107709, 2025.
- [13] C. Ragkhitwetsagul and J. Krinke, "Siamese: scalable and incremental code clone search via multiple code representations," *Empirical Software Engineering*, vol. 24, no. 4, pp. 2236–2284, 2019.
- [14] A. Fernández, S. Garcia, F. Herrera, and N. V. Chawla, "Smote for learning from imbalanced data: progress and challenges, marking the 15-year anniversary," *Journal of artificial intelligence research*, vol. 61, pp. 863–905, 2018.
- [15] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Ijcai*, vol. 14, no. 2, 1995, pp. 1137–1145.
- [16] P. Christen, D. J. Hand, and N. Kirielle, "A review of the f-measure: its history, properties, criticism, and alternatives," *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–24, 2023.
- [17] M. Buckland and F. Gey, "The relationship between recall and precision," *Journal of the American society for information science*, vol. 45, no. 1, pp. 12–19, 1994.
- [18] Ling, X. Charles, J. Huang, and H. Zhang, "Auc: a better measure than accuracy in comparing learning algorithms," in *Advances in artificial intelligence: 16th conference of the Canadian society for computational studies of intelligence, AI 2003*, vol. 16. Springer Berlin Heidelberg, June 2003, pp. 329–341.
- [19] A. Maćkiewicz and W. Ratajczak, "Principal components analysis (pca)," *Computers & Geosciences*, vol. 19, no. 3, pp. 303–342, 1993.
- [20] —, "Autoware overview," <https://autoware.org/autoware-overview/>, last accessed 2025/4/9.
- [21] Baidu, "Apolloautoaboutus," <https://www.apollo.auto/aboutus>, last accessed 2025/4/9.